CMP 519 Software Engineering

UNIT 05 – Design Patterns and Reuse

Instructor: Dr. Suresh Pokharel

DISCLAIMER

These slides are part of teaching materials for Software Engineering. These slides do not cover all aspect of learning Software Engineering, nor are these be taken as primary source of information. As the core textbooks and reference books for learning the subject has already been specified and provided to the students, students are encouraged to learn from the original sources.

Why Software Ruse?

- Definition: Reusing pre-existing software components to improve efficiency.
- Benefits:
  - Faster development cycles
  - Cost and effort reduction
  - Improved reliability and consistency
  - Standardization across projects

Challenges in Software Reuse

- Organizational Challenges:
  - Lack of a reuse culture
  - NIH Syndrome (Not Invented Here)

- Technical Challenges:
  - Difficulty in integrating components
  - Version control issues
  - Lack of proper documentation

- Economic Challenges:
  - High initial investment in reusable components
  - Cost of maintaining reusable libraries

Types of Software Reuse

1. White-Box Reuse (Code Reuse via Inheritance)
   - Access to internal implementation
   - Example: Extending a Java class

2. Black-Box Reuse (Component-Based)
   - Encapsulation hides implementation
   - Example: Using external libraries (e.g., logging framework)

3. Framework and Service Reuse
   - Example: Using Django framework for web applications

Understanding Solution Objects

- Objects designed to solve specific recurring problems.
- Example:
  - Sorting algorithms
  - Encryption libraries
  - Authentication modules
- Key Advantage: Solution objects enable standardized problem-solving.

Characteristics of Reusable Components

- ✅ Modular: Independent and loosely coupled
- ✅ Configurable: Easily adapted to different contexts
- ✅ Well-Documented: Clear usage instructions
- ✅ Versioned: Tracked for updates and maintenance

- Example: A reusable payment gateway module with multiple bank integrations.

Understanding Specification Inheritance

- When a subclass guarantees to behave like its superclass

```java
interface PaymentGateway {
    void processPayment(double amount);
}
class PayPalGateway implements PaymentGateway {
    public void processPayment(double amount) {
        // PayPal-specific implementation
    }
}
```

- Key Benefit: Enables polymorphism and improves maintainability.

Understanding Implementation Inheritance

Definition: A subclass reuses functionality from a superclass.

📌 **Example:** Extending an abstract class.

🔷 **Key Drawback:** Tight coupling, making future modifications harder.

💡 **Preferred Approach: Favor composition over inheritance** to reduce dependencies.

Liskov Substitution Principle (LSP)

- LSP states that subtypes must be substitutable for their base types without altering program correctness.
- Introduced by Barbara Liskov in 1987.

📌 Key Idea:

"If a program works correctly with a base class, it should also work correctly with any of its subclasses without modification."

- 📌 Why is it important?

✅ Ensures inheritance is used correctly

✅ Promotes reliable and maintainable code

✅ Helps in avoiding unexpected behavior in subclassing

Liskov Substitution Principle (LSP)

Common Violations of LSP:

- Overriding methods that break expected behavior
- Throwing unexpected exceptions
- Returning incompatible results

```
class Rectangle {
    int width, height;
    void setWidth(int w) { width = w; }
    void setHeight(int h) { height = h; }
    int getArea() { return width * height; }
}

class Square extends Rectangle {
    void setWidth(int w) { width = height = w; } // Violates LSP
    void setHeight(int h) { width = height = h; } // Violates LSP
}
```

Problem: A Square does not behave as a Rectangle—modifying one dimension should not affect the other. This breaks expected behavior.

Liskov Substitution Principle (LSP)

How to Fix LSP Violations?

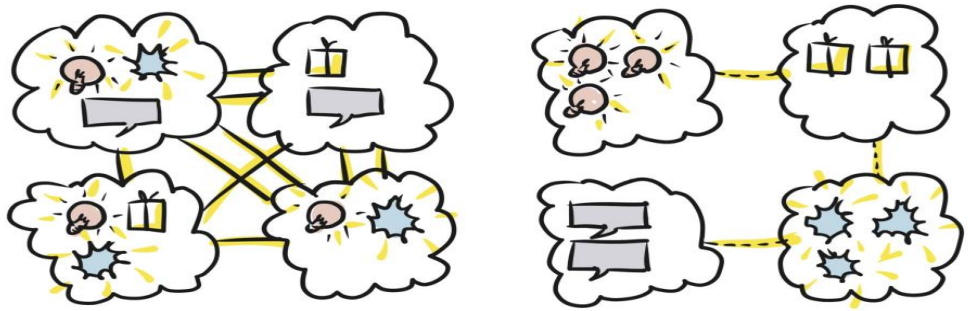
- Use composition over inheritance (Avoid forcing a Square to be a Rectangle)
- Redesign the hierarchy to maintain expected behavior

```
interface Shape {
    int getArea();
}

class Rectangle implements Shape {
    int width, height;
    public void setWidth(int w) { width = w; }
    public void setHeight(int h) { height = h; }
    public int getArea() { return width * height; }
}

class Square implements Shape {
    int side;
    public void setSide(int s) { side = s; }
    public int getArea() { return side * side; }
}
```

Now, both Rectangle and Square implement Shape, but their behaviors are correctly defined.

GENERAL CONCEPTS OF DESIGN PATTERNS

- Design Patterns are groups of objects (and their relationships) designed to support a 'good object design'
- What is 'good object design?'
  - design that yields high cohesion of our objects,
  - design that has low coupling between our objects, which, among other things, fosters more likely reuse

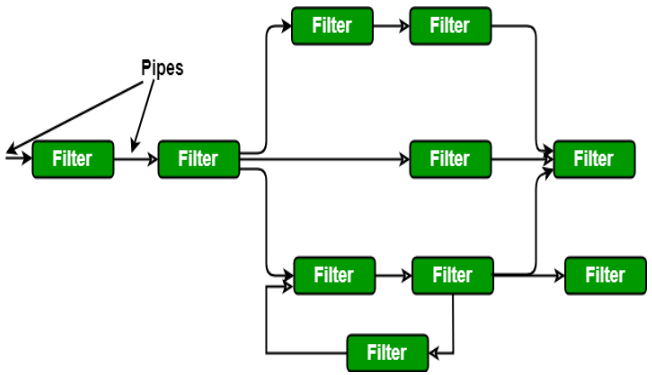


Strong coupling and low cohesion (on the left) vs. loose coupling and high cohesion (on the right).

PATTERNS - ARCHITECRURAL

Architectural Patterns:

- Related to large-scale and coarse-grained design;
- Typically applied during the early iterations (the elaboration phase) when the major structures and connections are established.
- Examples: pipe and filter; MVC; Client-Server; others…



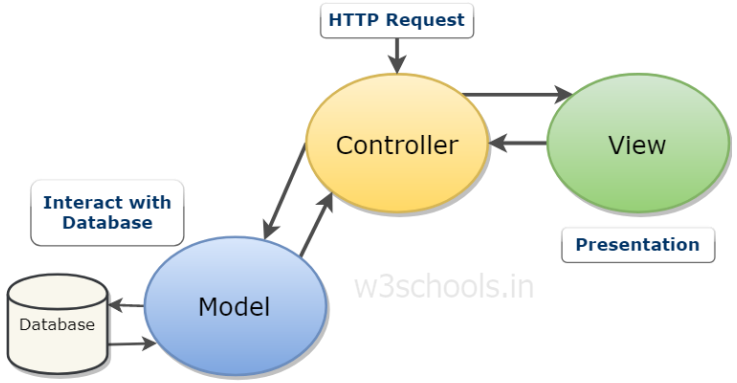
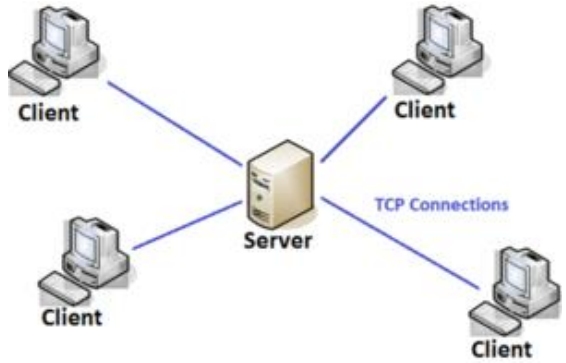


Fig: MVC Architecture



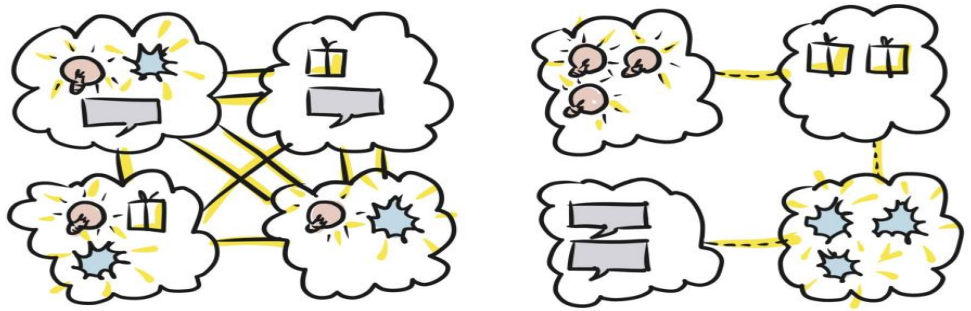
PATTERNS - DESIGN

Design Patterns:

- Related to the small and medium-scale design of objects and frameworks.
- "The critical design tool for software development is a mind well educated in design principles. It is not the UML or any other technology" - Criag Larman, applying UML and patterns

GENERAL CONCEPTS OF DESIGN PATTERNS

- Design Patterns are groups of objects (and their relationships) designed to support a 'good object design'
- What is 'good object design?'
  - design that yields high cohesion of our objects,
  - design that has low coupling between our objects, which, among other things, fosters more likely reuse



Strong coupling and low cohesion (on the left) vs. loose coupling and high cohesion (on the right).

GENERAL CONCEPTS OF DESIGN PATTERNS

- Good object design involves the assignment of object responsibilities.
- There are patterns or groupings of objects that work together to accommodate commonly needed activities / functions that subscribe to accepted principles of good design.
- Deciding what methods belong where and how objects interact (their relationships) is critically important and NOT trivial.
- All this is really at the heart of what it means to develop an object-oriented system, not merely the drawing of domain model diagrams, package diagrams, etc.

PATTERN DESCRIPTION PARAMETERS

Context:

- The general situation in which the pattern **applies**

Problem:

- A **short sentence** to raising the main difficulty.

Forces:

- The **issues or concerns** to consider when solving the problem

Solution:

- The **recommended way** to solve the problem in the **given context**.
  - 'to balance the forces'

Antipatterns: (Optional)

- Solutions that are inferior or do not work in this context.

Related patterns: (Optional)

- Patterns that are similar to this pattern.

References:

- Who developed or inspired the pattern.

CATEGORIES OF DESIGN PATTERENS

- There are several categories of patterns:
  - GRASP Patterns, and
  - Gang of Four (GoF) Patterns
  - Others too.

- In all cases, the patterns will be most helpful as we strive to design objects and groupings of objects to accommodate common issues.

RESPONSIBILITY

- Responsibility can be:
  - accomplished by a single object.
  - or a group of object collaboratively accomplish a responsibility.
- GRASP helps us in deciding which responsibility should be assigned to which object/class.
- Identify the objects and responsibilities from the problem domain, and also identify how objects interact with each other.
- Define blueprint for those objects – i.e., class with methods implementing those responsibilities.

RESPONSIBILITY - DOING AND KNOWING

- Responsibility is an obligation to perform a task or know information - Rebecca Wirfs-Brock
- Related to **behavior (doing)** and **data (knowing)**.
- Doing responsibility of an object is seen as:
  - doing something itself – create an object, process data, do some computation/calculation
  - initiate and coordinate actions with other objects
- Knowing responsibility of an object can be defined as:
  - private and public object data

GANG OF FOUR (GoF) PATTERNS

- Contributors: Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, with a foreword by Grady Booch
- GoF patterns categories:
  - Creational Design Patterns
  - Structural Design Patterns
  - Behavior Design Patterns

GANG OF FOUR (GoF) PATTERNS

- Creational Design Patterns
  - **Abstract Factory**. Allows the creation of objects without specifying their concrete type.
  - **Builder**. Uses to create complex objects.
  - **Factory Method. Creates objects without specifying the exact class to create.**
  - **Prototype**. Creates a new object from an existing object.
  - **Singleton**. Ensures only one instance of an object is created.

GANG OF FOUR (GoF) PATTERNS

- Structural Design Patterns
  - **Adapter**. Allows for two incompatible classes to work together by wrapping an interface around one of the existing classes.
  - **Bridge**. Decouples an abstraction so two classes can vary independently.
  - **Composite**. Takes a group of objects into a single object.
  - **Decorator**. Allows for an object's behavior to be extended dynamically at run time.
  - **Facade**. Provides a simple interface to a more complex underlying object.
  - **Flyweight**. Reduces the cost of complex object models.
  - **Proxy**. Provides a placeholder interface to an underlying object to control access, reduce cost, or reduce complexity.

GANG OF FOUR (GoF) PATTERNS

- Behavior Design Patterns
  - **Chain of Responsibility**. Delegates commands to a chain of processing objects.
  - **Command**. Creates objects which encapsulate actions and parameters.
  - **Interpreter**. Implements a specialized language.
  - **Iterator**. Accesses the elements of an object sequentially without exposing its underlying representation.
  - **Mediator**. Allows loose coupling between classes by being the only class that has detailed knowledge of their methods.
  - **Memento**. Provides the ability to restore an object to its previous state.
  - **Observer**. Is a publish/subscribe pattern which allows a number of observer objects to see an event.
  - **State**. Allows an object to alter its behavior when its internal state changes.
  - **Strategy**. Allows one of a family of algorithms to be selected on-the-fly at run-time.
  - **Template Method**. Defines the skeleton of an algorithm as an abstract class, allowing its sub-classes to provide concrete behavior.
  - **Visitor**. Separates an algorithm from an object structure by moving the hierarchy of methods into one object.

Let's explore all the design patterns from this website

https://refactoring.guru/design-patterns

Creational Patterns: Factory, Abstract Factory

Structural Patterns: Adapter, Composite, Bridge

Behavioral Patterns: Strategy, Command

Heuristics for Pattern Selection

📌 Challenges in Selecting Design Patterns

- Identifying the right pattern is difficult without experience.
- Large pattern catalogs make it impractical to read them entirely.
- Patterns address specific **design goals** and **nonfunctional requirements**.

📌 A Practical Approach

- Use **key phrases** from the **Requirements Analysis Document (RAD)** and **System Design Document (SDD)** to identify relevant patterns.

Heuristics for Pattern Selection

What problem are you solving?

✓ Does a pattern introduce unnecessary complexity?

✓ Will it improve maintainability and scalability?

🔷 **Example Decision-Making:**

- **Use Factory Method** when object creation logic varies.
- **Use Observer** when multiple components react to changes in one object.

Heuristics for Pattern Selection

Phrase	Design Pattern
• "Manufacturer independence" • "Platform independence"	Abstract Factory
• "Must comply with existing interface" • "Must reuse existing legacy component"	Adapter
• "Must support future protocols"	Bridge
• "All commands should be undoable" • "All transactions should be logged"	Command
• "Must support aggregate structures" • "Must allow for hierarchies of variable depth and width"	Composite
• "Policy and mechanisms should be decoupled". • "Must allow different algorithms to be interchanged at runtime."	Strategy

Integrating Design Patterns into Software

📌 **Why Integrate Design Patterns?**

- Patterns provide **proven solutions** to common design challenges.
- They **improve modularity, flexibility, and maintainability**.
- Proper integration ensures **scalability** and **reusability**.
- 📌 **Key Steps for Integration**

  1. **Identify Core Design Goals** – Determine primary requirements and constraints.
  2. **Select Suitable Patterns** – Use heuristics or key phrases from design documents.
  3. **Refine Architecture** – Adjust system design to fit chosen patterns.
  4. **Ensure Seamless Interaction** – Make sure patterns work well together.

Integrating Design Patterns into Software

Guidelines for Effective Integration

- ✅ **Use Multiple Patterns Together** – Combine patterns for comprehensive solutions (e.g., Composite + Iterator for collection traversal).
- ✅ **Avoid Overuse** – Only apply patterns when they **genuinely solve a problem**.
- ✅ **Maintain Loose Coupling** – Ensure components remain independent for flexibility.
- ✅ **Document Design Decisions** – Keep clear explanations of why specific patterns were chosen.
- ✅ **Refactor When Needed** – Adapt and refine designs as requirements evolve.

The Need for Reuse

- Software development has evolved from craft to engineering
- Hardware costs have dropped, but software costs have increased
- Systematic reuse helps reduce software costs
- Advantages of Reuse:
  - ✓ Lower development effort
  - ✓ Lower risk
  - ✓ Standardized terminology
  - ✓ Increased reliability

Challenges in Reuse

1 **NIH Syndrome (Not Invented Here):** Developers may prefer custom solutions over reusable ones.

2 **Lack of Process Support:** Identifying and customizing reusable solutions is not always straightforward.

3 **Training Needs:** Developers need education and guidance to effectively reuse components.

Documenting Reuse Strategies

Two key aspects:

1 **Documenting reusable solutions**

- Describe the solution and the problems it addresses
- Include trade-offs and alternative implementations
- Provide usage examples

2 **Documenting the system using reuse**

- Reference all reused solutions
- Maintain links between reused solutions and code
- Capture design context to reduce future modification costs

Assigning Responsibilities for Reuse

To encourage effective reuse, organizations must assign key roles:

- **Component Expert:** Knows a specific component deeply, provides guidance.
- **Pattern Expert:** Understands design patterns and best practices.
- **Technical Writer:** Documents dependencies between components and systems.
- **Configuration Manager:** Tracks versions and upgrades of reused components.

Encouraging a Reuse Culture

- Provide **access to expert developers**
- Offer **training on reusable components & patterns**
- Include **reuse-focused design reviews & code inspections**
- Ensure **documentation is regularly updated & accessible**
- Implement **knowledge-sharing initiatives**

END OF UNIT 05